AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 19.01.2027

Lab 14

Testy jednostkowe i TDD 🤖

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **co test dowodzi, a czego nie**
- Pisać testy w `unittest`; pracować w cyklu **TDD**
- Odcinać testy od świata zewnętrznego: **mocki**
- Mierzyć jakość testów: pokrycie i **testowanie mutacyjne**
- Efekty z sylabusa: **U1** (testy jednostkowe), **U3**, **KS1**



CZĘŚĆ I

Teoria: testowanie oprogramowania

Po co testy?

> *„Testowanie programu może wykazać obecność błędów, ale nigdy ich brak."* – E. W. Dijkstra (1970)

- Test to **wykonywalna specyfikacja**: zapisuje, jak kod ma się zachowywać
- Pozwala **bezpiecznie zmieniać** kod (refaktoryzacja, nowe funkcje)
- Każdy wasz lab był sprawdzany testami – dziś piszecie je sami

Piramida testów (M. Cohn):

POZIOM	CO SPRAWDZA	ILE ICH	SZYBKOŚĆ
jednostkowe	jedną funkcję w izolacji	najwięcej	milisekundy
integracyjne	współpracę modułów, bazę, API	mniej	sekundy
end-to-end	cały system jak użytkownik	najmniej	minuty

`unittest` w pigułce

```
import unittest
from arytmetyka import gcd

class TestGcd(unittest.TestCase):           # klasa = grupa testów
    def setUp(self):                          # przed KAŻDYM testem
        self.pairs = [(12, 18, 6), (17, 5, 1)]

    def test_typical(self):                   # metoda test_* = jeden test
        for a, b, expected in self.pairs:
            self.assertEqual(gcd(a, b), expected)

    def test_negative(self):
        self.assertEqual(gcd(-12, 18), 6)
```

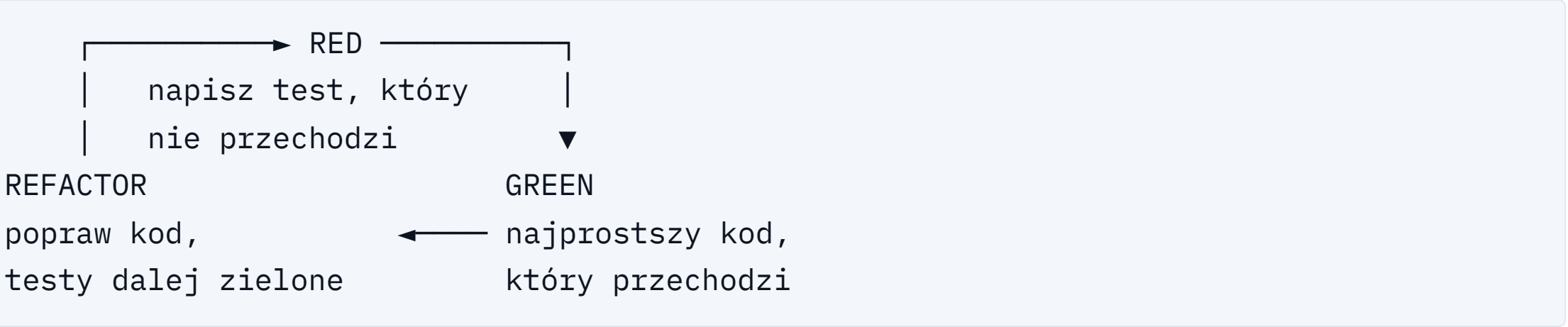
```
python -m unittest discover -s tests -v
```

Asercje i struktura testu

ASERCJA	SPRAWDZA
`assertEqual(a, b)`	`a == b`
`assertTrue(x)`, `assertFalse(x)`	prawdziwość
`assertAlmostEqual(a, b)`	liczby `float` z tolerancją
`assertIn(x, kolekcja)`	przynależność
`assertIsNone(x)`	`x is None`
`with assertRaises(ValueError):`	zgłoszenie wyjątku

AAA – Arrange, Act, Assert: przygotuj dane → wywołaj → sprawdź. Jeden test = jedno zachowanie, nazwa mówi, **co** jest sprawdzane: `test_empty_forecast_raises`.

TDD: Test-Driven Development



- Spopularyzował **Kent Beck** (*Test-Driven Development: By Example*, 2002)
- Zalety: kod od początku testowalny, małe kroki, wymagania zapisane jako testy
- Pułapka: testy pisane „pod implementację” zamiast pod **zachowanie**

Mocki: test bez internetu

```
from unittest import mock

def test_fetch_forecast_uses_mocked_http(self):
    with mock.patch.object(pogoda, "get_json", return_value=SAMPLE) as fake:
        forecast = pogoda.fetch_forecast(52.23, 21.01, 3)
    self.assertEqual(forecast[0]["temp"], -2.5)
    fake.assert_called_once_with(pogoda.build_url(52.23, 21.01, 3))
```

- **Atrapa** (*mock*) zastępuje zależność: sieć, bazę danych, zegar, losowość
- Test jest **szybki, powtarzalny** i nie zależy od pogody ani od tego, czy API działa
- `assert_called_once_with` sprawdza, **jak** kod użył zależności
- Ten sam pomysł co parametr `today` (lab 11) i `input_func` (lab 12)

Jak dobre są Twoje testy?

Pokrycie kodu (*coverage*): jaki procent linii wykonały testy

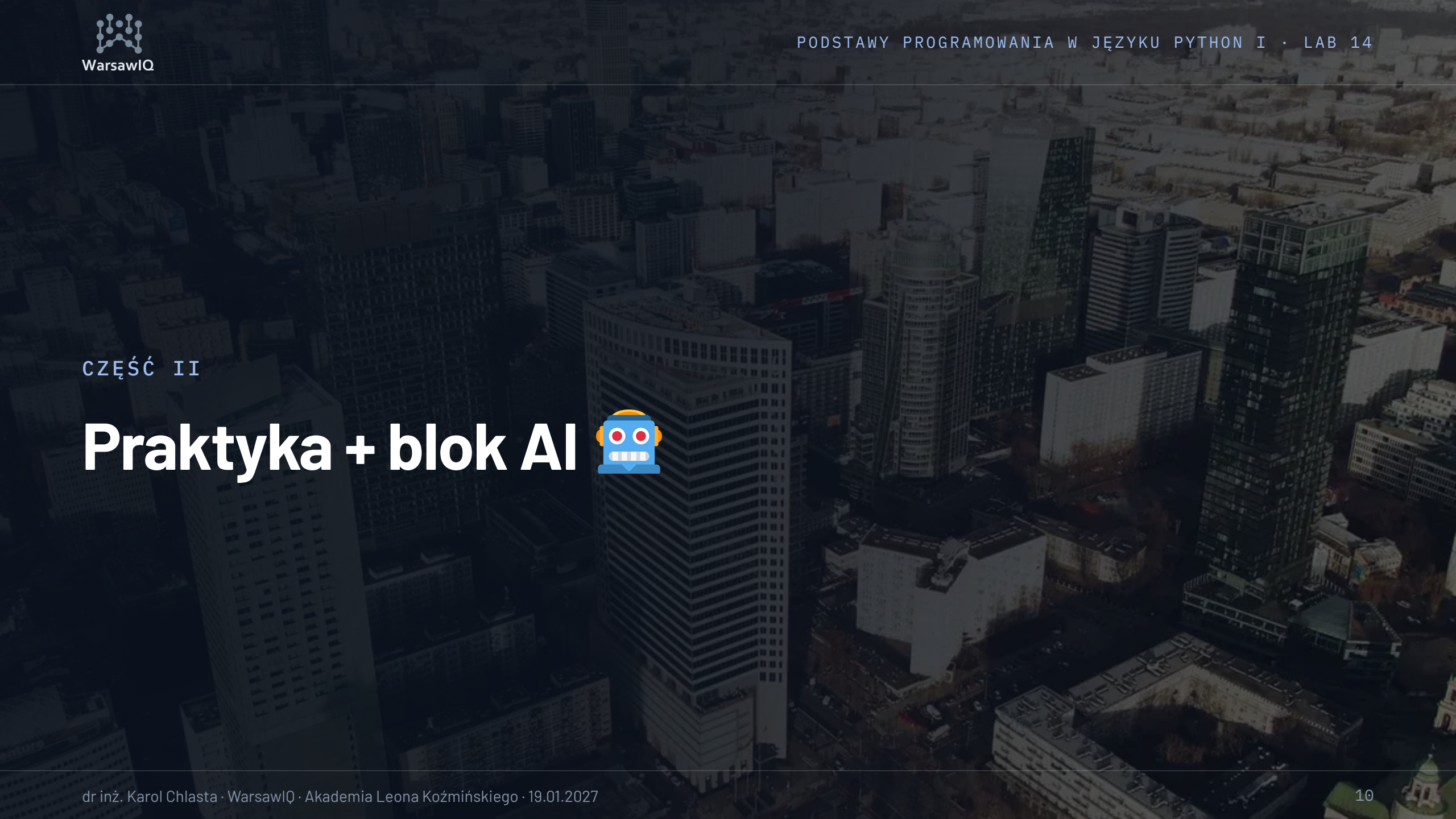
```
coverage run -m unittest discover -s tests && coverage report -m
```

- 100% pokrycia ≠ brak błędów: linia może się wykonać, a wynik nie być sprawdzony

Testowanie mutacyjne: wprowadź do kodu mały błąd (*mutanta*) i sprawdź, czy testy go wykryją

MUTANT	ZMIANA	ZABIJE GO TEST…
`n < 2` → `n < 1`	1 staje się „pierwsza”	`assertFalse(is_prime(1))`
`<=` → `<`	4, 9, 25 „pierwsze”	kwadraty liczb pierwszych

- **Wynik mutacyjny** = zabite mutanty / wszystkie – w zadaniu 2 musisz mieć 6/6



CZĘŚĆ II

Praktyka + blok AI 🤖

Blok AI: testy a AI

1. **Ty piszesz testy, AI implementuje** – czy AI spełnia testy uczciwie, czy wpisuje wyniki na sztywno?
2. **AI pisze testy, Ty oceniasz** – czy sprawdzają przypadki brzegowe, czy tylko „szczęśliwą ścieżkę”?
3. **AI wyjaśnia mocki** – zweryfikuj wyjaśnienie na naszym teście

> Testy to Twoja **specyfikacja** – AI może pomóc je pisać, ale to Ty decydujesz, jakie zachowanie jest poprawne.

Ciekawostki 🧪

- `unittest` (dawniej PyUnit) wzorowano na **JUnit** – frameworku Kenta Becka i Ericha Gammy dla Javy, który z kolei wywodził się z SUnit dla Smalltalka
- **Therac-25** (1985–1987): błędy w oprogramowaniu aparatu do radioterapii, m.in. wyścig przy szybkim wprowadzaniu danych, doprowadziły do śmiertelnych przedawkowań promieniowania
- **Knight Capital** (2012): błąd przy wdrożeniu reaktywował stary kod; firma straciła ok. **440 mln USD w 45 minut**
- W przemyśle popularny jest też **pytest** – testy jako zwykłe funkcje z `assert`; potrafi uruchamiać także testy `unittest`

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_pogoda.py`	aplikacja pogodowa w TDD + mock
★	`test_moje_arytmetyka.py`	Twoje testy – zabij 6 mutantów
🤖	`AI.md`	testy i AI
☆	—	`coverage`, własny test z mockiem

Sprawdzenie i oddanie

```
cd lab14_testy_tdd
python -m unittest test_moje_arytmetyka -v
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 14"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- 📝 **Kolokwium II: wtorek 26.01, 08:30–11:45, sala A/111** – zakres: laby 09–14

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 11
- K. Beck, *TDD. Sztuka tworzenia dobrego kodu*, Helion
- Dokumentacja: `unittest`, `unittest.mock` – *Getting Started*
- *coverage.py* (*coverage.readthedocs.io*)

Następne zajęcia (26.01, sala A/111): 📝 Kolokwium II